VECTOR

Hardware Specific Compare-and-Swap

Version 1.1.0
2022-11-23
Application Note AN-ISC-8-1230

Author Benjamin Seifert, Torsten Schmidt

Restrictions Customer Confidential – Vector decides

Abstract This document contains hardware specific example implementations of compare-and-swap

Table of Contents

1 Overview 2
2 Implementation Examples 2
2.1 ARMv7 2
2.2 Power PC 3
2.3 RH850 4
2.4 TriCore 5
2.5 Notes on Cache Coherence 5
3 Contacts 5

1 Overview

This document provides some sample implementations of compare-and-swap (CAS) for different platforms using hardware specific instructions.

Basically, compare-and-swap (CAS) is an atomic operation used for synchronization mechanisms which compares the content of a memory location with a given compare value (`oldVal`) and, if both are the same, sets the content of that memory location is to a given new value (`newVal`).

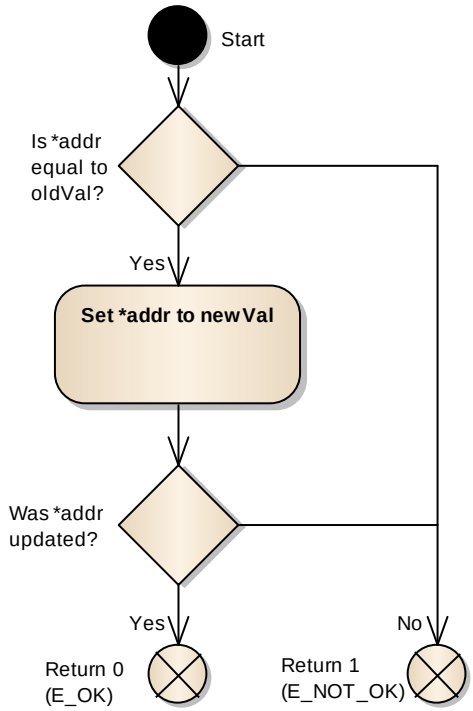


Figure 1-1 - CAS Activity Diagram

The examples shown are implemented for a dedicated compiler. If another compiler is used, adaptations might be necessary.

Caution
Vector Informatik GmbH does not warrant the correctness and proper functionality of the examples shown below.

2 Implementation Examples

2.1 ARMv7

The ARMv7 architecture provides the instructions `LDREX` and `STREX(EQ)` to ensure exclusive access to a specific address.

The following code example shows an implementation for the Green Hills Compiler using these instructions in Thumb execution mode.

For a detailed description of the instructions, please refer to the ARMv7 architecture and the specific hardware manual. Please also consider the following preconditions.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0



Example

```
__asm int CompareAndSwap(volatile int* addr, int oldVal, int newVal)
{
%reg addr %reg oldVal %reg newVal
  mov     r0, #1             /* prepare result r0 = 1 (E_NOT_OK) */
  ldrex   r2, [addr]         /* mark addr for exclusive access */
  cmp     r2, oldVal         /* compare addr with oldVal */
  itt     eq                 /* if equal, execute next 2 instructions */
  strexeq r2, newVal, [addr] /* write newVal to addr */
  moveq   r0, r2             /* result of strexeq to r0 (0 = E_OK) */
  /* data memory barrier, required after pair of ldrex and strex */
  dmb
%error
}
```



Caution

The following preconditions must be fulfilled to use the example on an ARMv7 hardware:

- The explicit derivative must support the instructions
- The given `addr` must be mapped to a memory location where these instructions are applicable (For details please refer to the specific hardware manual)
- The given `addr` must be within a "shareable" memory protection region, if memory protection is used

2.2 Power PC

The Power PC platform provides the instruction `stwdcbx` to ensure exclusive access to a specific address.

The following code example shows an implementation for the Green Hills Compiler using the instruction.

For a detailed description of the instruction, please refer to the corresponding hardware manual. Please also consider the following preconditions.



Example

```
__asm int CompareAndSwap(volatile int* addr, int oldVal, int newVal)
{
%reg addr %reg oldVal %reg newVal %lab cmpSwap_Return
  /* prepare decoration for Compare-and-Store instruction */
  e_lis   r5, 0x9000           /* prepare decoration value in r5 */
  se_or   oldVal, r5           /* rA = oldVal | r5 */
  /* stwdcbx -> Store Word Decorated with Cache Bypass */
  stwdcbx newVal, oldVal, addr /* compare addr with oldVal and
                                * set newVal */
  e_lwz   r6, 0x00(addr)       /* load result into r6 */
  se_li   r3, 0                /* prepare result r3 = 0 (E_OK) */
  cmpw    r6, newVal           /* check if newVal was written */
  e_beq   cmpSwap_Return       /* return if write was successful */
  se_li   r3, 1                /* return r3 = 1 (E_NOT_OK) */
cmpSwap_Return:
%error
}
```

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0



Caution
The following preconditions must be fulfilled to use the example on a Power PC hardware:

> The explicit derivative must support the instructions (DSMC module must exist)
> Only unsigned values with a maximum value of 2^28-1 can be processed

2.3 RH850

The RH850 platform provides the instructions `LDL.W` and `STC.W` to ensure exclusive access to a specific address.

The following code example shows an implementation for the Green Hills Compiler using these instructions.

For a detailed description of the instructions, please refer to the corresponding RH850 core manual. Please also consider the following preconditions.



Example

```
asm int CompareAndSwap(volatile int* addr, int oldVal, int newVal)
{
%reg addr %reg oldVal %reg newVal %lab cmpSwap_Return
  mov   1, r10          /* prepare result r10 = 1 (E_NOT_OK) */
  ldl.w [addr], r11     /* mark addr for exclusive access */
  cmp   oldVal, r11     /* compare addr with oldVal */
  bne   cmpSwap_Return /* return if not equal */
  stc.w newVal, [addr] /* else write newVal to addr */
                        /* if newVal = 1, addr was successfully updated */
  xori  1, newVal, r10 /* invert result of stc.w and return */
cmpSwap_Return:         /* return r10 (0 = E_OK) */
%error
}
```



Caution
The following preconditions must be fulfilled to use the example on a RH850 hardware:

> The exclusive access link to an address is deleted in 32-byte units. So ensure that only one mutual exclusion variable is allocated per 32 bytes of memory.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

2.4 TriCore

The TriCore platform provides the specific `CMPSWAP.W` instruction to ensure exclusive access to a specific address.

The following code example shows an implementation for the Tasking Compiler using the intrinsic function `__cmpswapw()`.

For a detailed description of the intrinsic function, please refer to the corresponding Compiler manual. Please also consider the following preconditions.



Caution
The following preconditions must be fulfilled to use the example on a TriCore hardware:

- The given `addr` must be word-aligned
- The given `addr` must lie within a range which has both read and write permissions enabled, if memory protection is used.
- When using the CMPSWAP instruction within a multicore environment the errata CPU_TC.H019 must be considered.



Example

```
int CompareAndSwap(volatile int* addr, int oldVal, int newVal)
{
  uint32 Retval = __cmpswapw((unsigned int *)addr, newVal, oldVal);
  if(Retval == oldVal)
  {
    /* addr has successfully been updated */
    Return E_OK;
  }
  else
  {
    /* addr has not been updated */
    Return E_NOT_OK;
  }
}
```

2.5 Notes on Cache Coherence



Caution
All variables which are accessed cross core must never be cached, otherwise you may run into cache coherency problems.

3 Contacts

For a full list with all Vector locations and addresses worldwide, please visit http://vector.com/contact/.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0